Pimpri-Chinchwad Education Trust's
**Pimpri-Chinchwad College of Engineering**
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University
(SPPU) ISO 21001:2018 Certified by TUV

# ASSIGNMENT 7: UNIVERSITY TIMETABLE SCHEDULING

**T.Y. B.TECH** | **Year: 2026–27** | **Semester: V**

**Department: Information Technology** | **Division: A** | **Course Code: BIT25PC04**

**Course: Design Analysis and Algorithm Laboratory**



| | |
|---|---|
| **NAME** | Sarvam Tarapure |
| **PRN** | 124B1F048 |



**GitHub Repository:** https://github.com/Black-sys-dot/Design-and-Analysis-of-Algorithms

## Scenario

A university must schedule examinations for courses whose students may be enrolled in multiple subjects. The timetable must avoid clashes, so a student is never assigned two exams in the same time slot. This assignment models the task as a graph-colouring problem.

Each course is represented by a vertex. An edge connects two courses whenever at least one student takes both. Assigning a colour to a vertex represents assigning an exam slot, and adjacent vertices must receive different colours.

### Requirements

1. Build the course-conflict graph and implement a colouring method such as Greedy Colouring or Backtracking.
2. Reduce the number of time slots while keeping the timetable conflict-free.
3. Consider scalability for thousands of courses and students.
4. Compare Greedy, DSATUR and Welsh–Powell strategies.
5. Extend the schedule with room capacity and availability constraints.

### Aim

Develop a practical exam-scheduling model using graph colouring that prevents student clashes and uses as few exam slots as possible.

### Course outcomes and objectives

Course outcomes: apply backtracking techniques and classify P, NP, NP-complete and NP-hard problems. Objectives: understand algorithmic complexity and choose suitable design strategies for real-world problems.

# Theory and Problem Representation

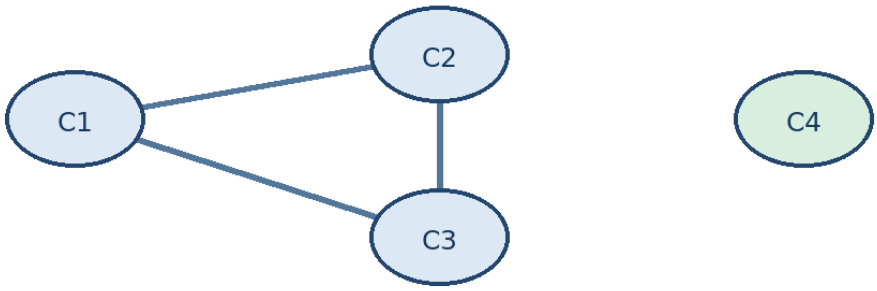
## 1. Graph theory

A graph G = (V, E) consists of vertices V and edges E. In this application, vertices stand for courses and edges encode a shared-student relationship. The graph is undirected because a conflict applies equally to both courses, and unweighted because the edge simply records whether a conflict exists.

## 2. Graph colouring and the conflict graph

Graph colouring assigns colours to vertices so that connected vertices never share a colour. Here, each colour corresponds to an exam time slot. The smallest number of colours that can colour the graph legally is its chromatic number.

To construct the graph, create one vertex for every course. For each student, connect every pair of courses in that student's enrolment list. Any pair joined by an edge must be scheduled at different times. This representation turns the timetable conflict rule into a graph constraint.




C1, C2 and C3 conflict pairwise; C4 has no shared-student conflict.

Illustration: C1, C2 and C3 share conflicts; C4 is not connected to them and can reuse a slot.

## 3. Colouring methods

**Greedy colouring:** Process vertices in a chosen order and assign each the smallest colour not already used by its coloured neighbours. It is straightforward and fast in practice, but the result depends on vertex order and may use more colours than optimal. Typical simple implementations can take $O(V^2)$.

**DSATUR:** Repeatedly select an uncoloured vertex with the greatest saturation degree, meaning the number of distinct colours already present among its neighbours. Degree is commonly used to break ties. This often yields strong solutions on difficult graphs; a straightforward implementation is commonly $O(V^2)$.

**Welsh–Powell:** Sort vertices by descending degree, then colour them greedily in that order. High-conflict courses are considered early. Its simple implementations are commonly $O(V^2)$, though exact cost depends on graph representation and implementation.

No heuristic guarantees a minimum colouring for every graph; the methods should be compared on the same conflict dataset.

# Room Allocation and Scalability

## 4. Room allocation constraints

Colouring determines when each exam can take place. A second pass assigns a room to each course within its slot. The room must have enough seats for the enrolled students, and each room can host at most one exam in a given slot. Choosing the smallest room that can accommodate an exam helps avoid wasting capacity.

Room assignment must respect both room capacity and availability. If no room fits a course in its assigned slot, the schedule needs more room capacity or a revised allocation; a colour assignment alone does not solve that resource constraint.

## 5. Scaling to large datasets

For large universities, use adjacency lists rather than a full adjacency matrix when the graph is sparse. Hash maps or dictionaries can map students to their enrolled courses and courses to their neighbours. Efficient sorting and graph traversal reduce overhead. Building conflict edges can also be parallelised carefully, followed by merging duplicate edges.

## 6. Core colouring update

For each course u, mark colours already used by its coloured neighbours as unavailable. Assign u the smallest remaining colour. If all existing colours are unavailable, introduce a new colour. This maintains the rule that no adjacent vertices share a time slot.

## Pseudocode: overall scheduling process

Input: courses C; students S and their course enrolments; rooms R with capacities.
Output: a mapping Course → Time Slot → Room.

```
BEGIN
  Create graph G with one vertex for each course in C
  FOR each student s in S:
      L = courses enrolled by s
      FOR each unordered pair (c1, c2) in L:
          add edge (c1, c2) to G

  Set colour[c] = UNASSIGNED for every course c
  Order courses by descending degree
  Colour the first course with colour 0
  FOR each remaining course u in that order:
      mark colours of coloured neighbours unavailable
      colour[u] = smallest available colour

  FOR each used time slot t:
      E = courses assigned to t
      order E by descending enrolment size
      order rooms by capacity
      FOR each course e in E:
          assign an available room with capacity >= enrolment(e)
          mark that room occupied for slot t

  FOR each course c:
      print c, colour[c], assigned_room[c]
END
```

# Worked Example and Output

## Example enrolments and room capacities

| Student | Enrolled courses |
|---|---|
| S1 | C1, C2 |
| S2 | C2, C3 |
| S3 | C1, C3 |
| S4 | C4 |
| **Room** | **Capacity** |
| R1 | 100 |
| R2 | 80 |
| R3 | 50 |

## One valid conflict-free timetable

| Course | Time slot | Room |
|---|---|---|
| C1 | Slot 1 | R1 |
| C2 | Slot 2 | R2 |
| C3 | Slot 3 | R3 |
| C4 | Slot 1 | R2 |

### Explanation

C1, C2 and C3 form a triangle in the conflict graph because each pair shares at least one student. Therefore, they need three distinct slots. C4 has no listed conflict and can reuse Slot 1. The room assignments are valid for the stated capacities, assuming the example enrolment counts fit those rooms and room availability is otherwise unrestricted.

## Algorithm comparison

| Method | Ordering rule | Strength / limitation |
|---|---|---|
| Greedy | Given or chosen order | Simple; colour count depends on order |
| DSATUR | Highest saturation first | Adapts to assigned colours; more selection overhead |
| Welsh–Powell | Descending vertex degree | Prioritises highly connected courses |

# Conclusion

Graph colouring provides a clear model for exam scheduling: courses become vertices, shared-student conflicts become edges, and colours represent exam slots. Greedy, DSATUR and Welsh–Powell offer different vertex-ordering strategies, with the number of slots depending on the graph and the chosen heuristic.

A separate room-allocation pass checks capacity and prevents a room from being used twice in the same slot. Adjacency lists and efficient student-course mappings help the approach scale to larger datasets. Together, these steps produce a practical timetable model that addresses both student conflicts and classroom resources.

## Key points to verify in an implementation

- Every shared-student course pair must have an edge.
- Adjacent courses must have different colours.
- Each assigned room must fit the course enrolment.
- A room cannot host two exams in one time slot.
- If room allocation fails, the schedule must be revised rather than silently leaving a course unassigned.

End of Assignment 7